Physlib Monthly Report

1–31 December 2025

Contributors: Davood Haji Taghi Tehrani, Fabio Anza, Joseph Tooby-Smith, Miyahara Kō, Nicola Bernini, Pietro Monticone, rht, Shlok Vaibhav Singh

Reviewers: Joseph Tooby-Smith

Generated 14 August 2026 from commit `7ab2194`.

Abstract

Physlib is a community library of formalised physics for the Lean 4 proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month's additions to it.

It covers 1–31 December 2025 on branch `master` of `leanprover-community/physlib`, between commits `b47abf9` and `7ab2194`. Over that period 8 contributors landed 26 commits, changing 6,709 lines across 138 files and adding 133 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at GitHub compare.

Contents

1 Introduction

During Dec 2025, 8 contributors submitted 26 commits that changed 6,709 lines (+3,691 / -3,018) across 138 files spanning 23 top-level areas of the repository. Of those, 5 files were newly added and 4 were removed outright. This report catalogues 133 newly added Lean declarations: 107 lemmas, 20 defs, 3 theorems, 1 abbrev, 1 structure, and 1 instance. We're delighted to recognize the following first-time contributors this month: Davood Haji Taghi Tehrani, Fabio Anza, and rht.

See the full compare diff and the list of commits on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Davood Haji Taghi Tehrani	94	1
Fabio Anza	213	1
Joseph Tooby-Smith	5,356	10
Miyahara Kō	9	1
Nicola Bernini	677	7
Pietro Monticone	6	2
rht	115	3
Shlok Vaibhav Singh	309	1

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Joseph Tooby-Smith	14

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
`PhysLean/Electromagnetism`	18	+2	-4	+1,816	-1,773
`PhysLean/Particles`	11	+1	–	+278	-299
`PhysLean/SpaceAndTime`	15	–	–	+378	-87
`PhysLean/Relativity`	17	–	–	+320	-113
`PhysLean/QFT`	28	–	–	+166	-239
`PhysLean/Mathematics`	13	–	–	+125	-262
`PhysLean/ClassicalMechanics`	8	–	–	+245	-107
`PhysLean/Thermodynamics`	2	+1	–	+221	-8
`PhysLean/StatisticalMechanics`	1	–	–	+54	-66
`PhysLean/QuantumMechanics`	4	–	–	+36	-21
`lake-manifest.json`	1	–	–	+16	-16
`scripts/MetaPrograms`	7	–	–	+10	-11
`.codespellignore`	1	+1	–	+10	-0

PhysLean.lean	1	–	–	+4	-4
PhysLean/StringTheory	2	–	–	+2	-4
lakefile.toml	1	–	–	+2	-2
PhysLean/Meta	2	–	–	+2	-1
PhysLean/CondensedMatter	1	–	–	+1	-1
PhysLean/Units	1	–	–	+1	-1
README.md	1	–	–	+1	-1
lean-toolchain	1	–	–	+1	-1
scripts	1	–	–	+1	-1
PhysLean/Cosmology	1	–	–	+1	-0

1.4 New declarations by kind

Kind	Count
lemma	107
def	20
theorem	3
abbrev	1
structure	1
instance	1

2 Details by file

2.1 PhysLean.ClassicalMechanics.Pendulum.CoplanarDoublePendulum

PhysLean/ClassicalMechanics/Pendulum/CoplanarDoublePendulum.lean on GitHub.

def ConfigurationSpace [source]

The configuration space of the coplaner double pendulum.

```
@[sorryful]
def ConfigurationSpace : Type
```

2.2 PhysLean.ClassicalMechanics.Pendulum.SlidingPendulum

PhysLean/ClassicalMechanics/Pendulum/SlidingPendulum.lean on GitHub.

def ConfigurationSpace [source]

The configuration space of the sliding pendulum system.

```
@[sorryful]
def ConfigurationSpace : Type
```

2.3 PhysLean.Electromagnetism.Current.InfiniteWire

PhysLean/Electromagnetism/Current/InfiniteWire.lean on GitHub.

def wireCurrentDensity [source]

The current density associated with an infinite wire carrying a current `I` along the `x`-axis.

```
noncomputable def wireCurrentDensity (c : SpeedOfLight) :
    ℝ →ₗ[ℝ] DistLorentzCurrentDensity 3 where
```

lemma wireCurrentDensity_chargeDesnity [source]

```
@[simp]
lemma wireCurrentDensity_chargeDesnity (c : SpeedOfLight) (I : ℝ) :
    (wireCurrentDensity c I).chargeDensity c = 0
```

lemma wireCurrentDensity_currentDensity_fst [source]

```
lemma wireCurrentDensity_currentDensity_fst (c : SpeedOfLight) (I : ℝ)
    (η : 𝓢(Time × Space 3, ℝ)) :
    (wireCurrentDensity c I).currentDensity c η 0 =
    (constantTime <|
    constantSliceDist 0 <|
    I • diracDelta ℝ 0) η
```

lemma wireCurrentDensity_currentDensity_snd [source]

```
@[simp]
lemma wireCurrentDensity_currentDensity_snd (c : SpeedOfLight) (I : ℝ)
    (ε : 𝓢(Time × Space 3, ℝ)) :
    (wireCurrentDensity c I).currentDensity c ε 1 = 0
```

lemma wireCurrentDensity_currentDensity_thrd [source]

```
@[simp]
lemma wireCurrentDensity_currentDensity_thrd (c : SpeedOfLight) (I : ℝ)
    (ε : 𝓢(Time × Space 3, ℝ)) :
    (wireCurrentDensity c I).currentDensity c ε 2 = 0
```

def infiniteWire [source]

The electromagnetic potential of an infinite wire along the x-axis carrying a current `I`.

```
noncomputable def infiniteWire (𝓕 : FreeSpace) (I : ℝ) :
    DistElectromagneticPotential 3
```

lemma infiniteWire_scalarPotential [source]

```
@[simp]
lemma infiniteWire_scalarPotential (𝓕 : FreeSpace) (I : ℝ) :
    (infiniteWire 𝓕 I).scalarPotential 𝓕.c = 0
```

lemma infiniteWire_vectorPotential [source]

```
lemma infiniteWire_vectorPotential (𝓕 : FreeSpace) (I : ℝ) :
    (infiniteWire 𝓕 I).vectorPotential 𝓕.c =
    (constantTime <|
    constantSliceDist 0
    ((- I * 𝓕.μ₀ / (2 * Real.pi)) • distOfFunction (fun (x : Space 2) =>
      Real.log ‖x‖ • EuclideanSpace.single 0 (1 : ℝ))
    (IsDistBounded.log_norm.smul_const _)))
```

lemma infiniteWire_vectorPotential_fst [source]

```
lemma infiniteWire_vectorPotential_fst (𝓕 : FreeSpace) (I : ℝ)(η : 𝓢(Time × Space 3, ℝ)) :
    (infiniteWire 𝓕 I).vectorPotential 𝓕.c η 0 =
    (constantTime <|
    constantSliceDist 0 <|
    (- I * 𝓕.μ₀ / (2 * Real.pi)) • distOfFunction (fun (x : Space 2) => Real.log ‖x‖)
    (IsDistBounded.log_norm)) η
```

lemma infiniteWire_vectorPotential_snd [source]

```
@[simp]
lemma infiniteWire_vectorPotential_snd (𝓕 : FreeSpace) (I : ℝ) :
    (infiniteWire 𝓕 I).vectorPotential 𝓕.c η 1 = 0
```

lemma infiniteWire_vectorPotential_thrd [source]

```
@[simp]
lemma infiniteWire_vectorPotential_thrd (𝓕 : FreeSpace) (I : ℝ) :
    (infiniteWire 𝓕 I).vectorPotential 𝓕.c η 2 = 0
```

lemma infiniteWire_vectorPotential_distTimeDeriv [source]

```
@[simp]
lemma infiniteWire_vectorPotential_distTimeDeriv (𝓕 : FreeSpace) (I : ℝ) :
    distTimeDeriv ((infiniteWire 𝓕 I).vectorPotential 𝓕.c) = 0
```

lemma infiniteWire_vectorPotential_distSpaceDeriv_0 [source]

```
@[simp]
lemma infiniteWire_vectorPotential_distSpaceDeriv_0 (𝓕 : FreeSpace) (I : ℝ) :
    distSpaceDeriv 0 ((infiniteWire 𝓕 I).vectorPotential 𝓕.c) = 0
```

lemma infiniteWire_electricField [source]

```
@[simp]
lemma infiniteWire_electricField (𝓕 : FreeSpace) (I : ℝ) :
    (infiniteWire 𝓕 I).electricField 𝓕.c = 0
```

lemma infiniteWire_isExterma [source]

```
lemma infiniteWire_isExterma {𝓕 : FreeSpace} {I : ℝ} :
    IsExtrema 𝓕 (infiniteWire 𝓕 I) (wireCurrentDensity 𝓕.c I)
```

2.4 PhysLean.Electromagnetism.Dynamics.CurrentDensity

PhysLean/Electromagnetism/Dynamics/CurrentDensity.lean on GitHub.

abbrev DistLorentzCurrentDensity [source]

The Lorentz current density, also called four-current as a distribution.

```
abbrev DistLorentzCurrentDensity (d : ℕ := 3)
```

def chargeDensity [source]

The charge density underlying a Lorentz current density which is a distribution.

```
noncomputable def chargeDensity {d : ℕ} (c : SpeedOfLight) :
    (DistLorentzCurrentDensity d) →ₗ[ℝ] (Time × Space d) →d[ℝ] ℝ where
```

def currentDensity [source]

The underlying (non-Lorentz) current density associated with a distributive Lorentz current density.

```
noncomputable def currentDensity (c : SpeedOfLight) :
    DistLorentzCurrentDensity d →ₗ[ℝ] (Time × Space d) →d[ℝ] EuclideanSpace ℝ (Fin d) where
```

2.5 PhysLean.Electromagnetism.Dynamics.IsExtrema

PhysLean/Electromagnetism/Dynamics/IsExtrema.lean on GitHub.

lemma isExtrema_iff_tensors [source]

```
lemma isExtrema_iff_tensors {𝓕 : FreeSpace}
    (A : ElectromagneticPotential d)
    (hA : ContDiff ℝ ∞ A) (J : LorentzCurrentDensity d) (hJ : ContDiff ℝ ∞ J) :
    IsExtrema 𝓕 A J ↔ ∀ x,
    {((1/ 𝓕.μ₀ : ℝ) • tensorDeriv A.toFieldStrength x | κ κ ν') + - (J x | ν')}^T = 0
```

lemma isExtrema_lorentzGroup_apply_iff [source]

```
lemma isExtrema_lorentzGroup_apply_iff {𝓕 : FreeSpace}
    (A : ElectromagneticPotential d)
    (hA : ContDiff ℝ ∞ A) (J : LorentzCurrentDensity d) (hJ : ContDiff ℝ ∞ J)
    (Λ : LorentzGroup d) :
    IsExtrema 𝓕 (fun x => Λ • A (Λ⁻¹ • x)) (fun x => Λ • J (Λ⁻¹ • x)) ↔
    IsExtrema 𝓕 A J
```

def IsExtrema [source]

The proposition on an electromagnetic potential, corresponding to the statement that it is an extrema of the lagrangian.

```
def IsExtrema {d} (𝓕 : FreeSpace)
    (A : DistElectromagneticPotential d)
    (J : DistLorentzCurrentDensity d) : Prop
```

lemma isExtrema_iff_gradLagrangian [source]

```
lemma isExtrema_iff_gradLagrangian {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d)
    (J : DistLorentzCurrentDensity d) :
    IsExtrema 𝓕 A J ↔ A.gradLagrangian 𝓕 J = 0
```

lemma isExtrema_iff_components [source]

```
lemma isExtrema_iff_components {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d)
    (J : DistLorentzCurrentDensity d) :
    IsExtrema 𝓕 A J ↔ (∀ ε, A.gradLagrangian 𝓕 J ε (Sum.inl 0) = 0)
    ∧ (∀ ε i, A.gradLagrangian 𝓕 J ε (Sum.inr i) = 0)
```

lemma isExtrema_iff_space_time [source]

```
lemma isExtrema_iff_space_time {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d)
    (J : DistLorentzCurrentDensity d) :
    IsExtrema 𝓕 A J ↔
      (∀ ε, distSpaceDiv (A.electricField 𝓕.c) ε = (1/𝓕.ε₀) * (J.chargeDensity 𝓕.c) ε) ∧
      (∀ ε i, 𝓕.μ₀ * 𝓕.ε₀ * (Space.distTimeDeriv (A.electricField 𝓕.c)) ε i -
      ∑ j, ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
        ((Space.distSpaceDeriv j (A.magneticFieldMatrix 𝓕.c)) ε) (j, i) +
      𝓕.μ₀ * J.currentDensity 𝓕.c ε i = 0)
```

lemma isExtrema_iff_vectorPotential [source]

```
lemma isExtrema_iff_vectorPotential {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d)
    (J : DistLorentzCurrentDensity d) :
    IsExtrema 𝓕 A J ↔
      (∀ ε, distSpaceDiv (A.electricField 𝓕.c) ε = (1/𝓕.ε₀) * (J.chargeDensity 𝓕.c) ε) ∧
      (∀ ε i, 𝓕.μ₀ * 𝓕.ε₀ * distTimeDeriv (A.electricField 𝓕.c) ε i -
      (∑ x, -(distSpaceDeriv x (distSpaceDeriv x (A.vectorPotential 𝓕.c)) ε i
        - distSpaceDeriv x (distSpaceDeriv i (A.vectorPotential 𝓕.c)) ε x)) +
      𝓕.μ₀ * J.currentDensity 𝓕.c ε i = 0)
```

lemma isExterma_iff_tensor [source]

```
lemma isExterma_iff_tensor {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d)
    (J : DistLorentzCurrentDensity d) :
    IsExtrema 𝓕 A J ↔ ∀ ε,
    {((1/ 𝓕.μ₀ : ℝ) • distTensorDeriv A.fieldStrength ε | κ κ ν') + - (J ε | ν')}^T = 0
```

lemma isExterma_equivariant [source]

```
lemma isExterma_equivariant {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d)
    (J : DistLorentzCurrentDensity d) (Λ : LorentzGroup d) :
    IsExtrema 𝓕 (Λ • A) (Λ • J) ↔ IsExtrema 𝓕 A J
```

2.6 PhysLean.Electromagnetism.Dynamics.KineticTerm

PhysLean/Electromagnetism/Dynamics/KineticTerm.lean on GitHub.

lemma gradKineticTerm_eq_tensorDeriv [source]

```
lemma gradKineticTerm_eq_tensorDeriv {d} {𝓕 : FreeSpace}
    (A : ElectromagneticPotential d) (x : SpaceTime d)
    (hA : ContDiff ℝ ∞ A) (ν : Fin 1 ⊕ Fin d) :
    A.gradKineticTerm 𝓕 x ν = η ν ν * ((Tensorial.toTensor (M := Lorentz.Vector d)).symm
    (permT id (PermCond.auto) {(1/ 𝓕.μ₀ : ℝ) • tensorDeriv A.toFieldStrength x | κ κ ν'}^T)) ν
```

def gradKineticTerm [source]

The gradient of the kinetic term for an Electromagnetic potential which is a distribution.

```
noncomputable def gradKineticTerm {d} (𝓕 : FreeSpace) :
    DistElectromagneticPotential d →ₗ[ℝ] (SpaceTime d) →d[ℝ] Lorentz.Vector d where
```

lemma gradKineticTerm_eq_sum_sum [source]

```
lemma gradKineticTerm_eq_sum_sum {d} {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) :
    A.gradKineticTerm 𝓕 ε = ∑ ν, ∑ μ,
        (1 / (𝓕.μ₀) * (η μ μ * η ν ν * distDeriv μ (distDeriv μ A) ε ν -
        distDeriv μ (distDeriv ν A) ε μ)) • Lorentz.Vector.basis ν
```

lemma gradKineticTerm_eq_fieldStrength [source]

```
lemma gradKineticTerm_eq_fieldStrength {d} {𝓕 : FreeSpace} (A : DistElectromagneticPotential d
    )
    (ε : 𝓢(SpaceTime d, ℝ)) :
    A.gradKineticTerm 𝓕 ε = ∑ ν, (1/𝓕.μ₀ * η ν ν) •
    (∑ μ, ((Vector.basis.tensorProduct Vector.basis).repr
      (distDeriv μ (A.fieldStrength) ε) (μ, ν))) • Lorentz.Vector.basis ν
```

lemma gradKineticTerm_sum_inl_eq [source]

```
lemma gradKineticTerm_sum_inl_eq {d} {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) :
    A.gradKineticTerm 𝓕 ε (Sum.inl 0) =
    (1/(𝓕.μ₀ * 𝓕.c) * (distTimeSlice 𝓕.c).symm (Space.distSpaceDiv (A.electricField 𝓕.c)) ε)
```

lemma gradKineticTerm_sum_inr_eq [source]

```
lemma gradKineticTerm_sum_inr_eq {d} {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) (i : Fin d) :
    A.gradKineticTerm 𝓕 ε (Sum.inr i) =
    (𝓕.μ₀⁻¹ * (1 / 𝓕.c ^ 2 * (distTimeSlice 𝓕.c).symm
      (Space.distTimeDeriv (A.electricField 𝓕.c)) ε i -
      ∑ j, ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
        ((distTimeSlice 𝓕.c).symm (Space.distSpaceDeriv j
          (A.magneticFieldMatrix 𝓕.c)) ε) (j, i)))
```

lemma gradKineticTerm_eq_distTensorDeriv [source]

```
lemma gradKineticTerm_eq_distTensorDeriv {d} {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) (ν : Fin 1 ⊕ Fin d) :
    A.gradKineticTerm 𝓕 ε ν = η ν ν * ((Tensorial.toTensor (M := Lorentz.Vector d)).symm
    (permT id (PermCond.auto) {(1/ 𝓕.μ₀ : ℝ) •
    distTensorDeriv A.fieldStrength ε | κ κ ν'}ᵀ)) ν
```

2.7 PhysLean.Electromagnetism.Dynamics.Lagrangian

PhysLean/Electromagnetism/Dynamics/Lagrangian.lean on GitHub.

lemma ElectromagneticPotential.gradFreeCurrentPotential_eq_tensor [source]

```
lemma gradFreeCurrentPotential_eq_tensor {d} (A : ElectromagneticPotential d)
    (hA : ContDiff ℝ ∞ A) (J : LorentzCurrentDensity d)
    (hJ : ContDiff ℝ ∞ J) (x : SpaceTime d) (ν : Fin 1 ⊕ Fin d) :
    A.gradFreeCurrentPotential J x ν = η ν ν * ((Tensorial.toTensor (M := Lorentz.Vector d)).
    symm
    (permT id (PermCond.auto) {J x | ν'}ᵀ)) ν
```

lemma ElectromagneticPotential.gradLagrangian_eq_tensor [source]

```
lemma gradLagrangian_eq_tensor {𝓕 : FreeSpace}
    (A : ElectromagneticPotential d)
    (hA : ContDiff ℝ ∞ A) (J : LorentzCurrentDensity d)
    (hJ : ContDiff ℝ ∞ J) (x : SpaceTime d) (ν : Fin 1 ⊕ Fin d) :
    A.gradLagrangian 𝓕 J x ν =
    η ν ν * ((Tensorial.toTensor (M := Lorentz.Vector d)).symm
    (permT id (PermCond.auto) {((1/ 𝓕.μ₀ : ℝ) • tensorDeriv A.toFieldStrength x | κ κ ν') +
    - (J x | ν')}ᵀ)) ν
```

def gradFreeCurrentPotential [source]

The variational gradient of the free current potential for distributional potentials.

```
noncomputable def gradFreeCurrentPotential {d} :
    DistLorentzCurrentDensity d →ₗ[ℝ] ((SpaceTime d) →d[ℝ] Lorentz.Vector d) where
```

lemma gradFreeCurrentPotential_eq_sum_basis [source]

```
lemma gradFreeCurrentPotential_eq_sum_basis {d}
    (J : DistLorentzCurrentDensity d) (ε : 𝓢(SpaceTime d, ℝ)) :
    (gradFreeCurrentPotential J) ε =
    (∑ μ, (η μ μ • (J ε μ) • Lorentz.Vector.basis μ))
```

lemma gradFreeCurrentPotential_sum_inl_0 [source]

```
lemma gradFreeCurrentPotential_sum_inl_0 (𝓕 : FreeSpace) {d}
    (J : DistLorentzCurrentDensity d) (ε : 𝓢(SpaceTime d, ℝ)) :
    (gradFreeCurrentPotential J) ε (Sum.inl 0) =
    𝓕.c * (distTimeSlice 𝓕.c).symm (J.chargeDensity 𝓕.c) ε
```

lemma gradFreeCurrentPotential_sum_inr_i [source]

```
lemma gradFreeCurrentPotential_sum_inr_i (𝓕 : FreeSpace) {d}
    (J : DistLorentzCurrentDensity d) (ε : 𝓢(SpaceTime d, ℝ)) (i : Fin d) :
    (gradFreeCurrentPotential J) ε (Sum.inr i) =
    - (distTimeSlice 𝓕.c).symm (J.currentDensity 𝓕.c) ε i
```

lemma DistElectromagneticPotential.gradFreeCurrentPotential_eq_tensor [source]

```
lemma gradFreeCurrentPotential_eq_tensor {d}
    (J : DistLorentzCurrentDensity d) (ε : 𝓢(SpaceTime d, ℝ))
    (ν : Fin 1 ⊕ Fin d) :
    gradFreeCurrentPotential J ε ν = η ν ν * ((Tensorial.toTensor (M := Lorentz.Vector d)).symm
    (permT id (PermCond.auto) {J ε | ν'}ᵀ)) ν
```

def gradLagrangian [source]

The variational gradient of lagrangian for an electromagnetic potential which is a distribution.

```
noncomputable def gradLagrangian {d} (𝓕 : FreeSpace) (A : DistElectromagneticPotential d)
    (J : DistLorentzCurrentDensity d) : ((SpaceTime d) →d[ℝ] Lorentz.Vector d)
```

lemma gradLagrangian_sum_inl_0 [source]

```
lemma gradLagrangian_sum_inl_0 {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d) (J : DistLorentzCurrentDensity d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    A.gradLagrangian 𝓕 J ε (Sum.inl 0) =
    (1/(𝓕.μ₀ * 𝓕.c) * (distTimeSlice 𝓕.c).symm (Space.distSpaceDiv (A.electricField 𝓕.c)) ε)
    - 𝓕.c * (distTimeSlice 𝓕.c).symm (J.chargeDensity 𝓕.c) ε
```

lemma gradLagrangian_sum_inr_i [source]

```
lemma gradLagrangian_sum_inr_i {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d) (J : DistLorentzCurrentDensity d)
    (ε : 𝓢(SpaceTime d, ℝ)) (i : Fin d) :
    A.gradLagrangian 𝓕 J ε (Sum.inr i) =
    𝓕.μ₀⁻¹ * (1 / 𝓕.c ^ 2 *
      (distTimeSlice 𝓕.c).symm (Space.distTimeDeriv (A.electricField 𝓕.c)) ε i -
      ∑ j, ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
        ((distTimeSlice 𝓕.c).symm (Space.distSpaceDeriv j (A.magneticFieldMatrix 𝓕.c)) ε) (j,
    i)) +
    (distTimeSlice 𝓕.c).symm (J.currentDensity 𝓕.c) ε i
```

lemma DistElectromagneticPotential.gradLagrangian_eq_tensor [source]

```
lemma gradLagrangian_eq_tensor {𝓕 : FreeSpace}
    (A : DistElectromagneticPotential d) (J : DistLorentzCurrentDensity d)
    (ε : 𝓢(SpaceTime d, ℝ)) (ν : Fin 1 ⊕ Fin d) :
    A.gradLagrangian 𝓕 J ε ν =
    η ν ν * ((Tensorial.toTensor (M := Lorentz.Vector d)).symm
    (permT id (PermCond.auto) {((1/ 𝓕.μ₀ : ℝ) • distTensorDeriv A.fieldStrength ε | κ κ ν') +
    - (J ε | ν')}ᵀ)) ν
```

2.8 PhysLean.Electromagnetism.Kinematics.ElectricField

PhysLean/Electromagnetism/Kinematics/ElectricField.lean on GitHub.

lemma electricField_eq_fieldStrength [source]

```
lemma electricField_eq_fieldStrength {d} {c : SpeedOfLight}
    (A : DistElectromagneticPotential d) (ε : 𝓢(Time × Space d, ℝ))
    (i : Fin d) : A.electricField c ε i = - c * (Vector.basis.tensorProduct Vector.basis).repr
      (distTimeSlice c (A.fieldStrength) ε) (Sum.inl 0, Sum.inr i)
```

2.9 PhysLean.Electromagnetism.Kinematics.EMPotential

PhysLean/Electromagnetism/Kinematics/EMPotential.lean on GitHub.

lemma deriv_eq_sum_sum [source]

```
lemma deriv_eq_sum_sum {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    deriv A ε =∑ μ, ∑ ν, (SpaceTime.distDeriv μ A ε ν) •
      Lorentz.CoVector.basis μ ⊗ₜ[ℝ] Lorentz.Vector.basis ν
```

2.10 PhysLean.Electromagnetism.Kinematics.FieldStrength

PhysLean/Electromagnetism/Kinematics/FieldStrength.lean on GitHub.

lemma fieldStrengthMatrix_eq [source]

```
lemma fieldStrengthMatrix_eq {d} (A : ElectromagneticPotential d) (x : SpaceTime d) :
    A.fieldStrengthMatrix x =
    (Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr (A.toFieldStrength x)
```

lemma toFieldStrength_eq_fieldStrengthMatrix [source]

```
lemma toFieldStrength_eq_fieldStrengthMatrix {d} (A : ElectromagneticPotential d) :
    toFieldStrength A = fun x => ∑ μ, ∑ ν,
      A.fieldStrengthMatrix x (μ, ν) • (Lorentz.Vector.basis μ) ⊗ₜ (Lorentz.Vector.basis ν)
```

lemma toFieldStrength_differentiable [source]

```
lemma toFieldStrength_differentiable {d} {A : ElectromagneticPotential d}
    (hA : ContDiff ℝ 2 A) :
    Differentiable ℝ (toFieldStrength A)
```

lemma fieldStrength_eq_fieldStrengthAux [source]

```
lemma fieldStrength_eq_fieldStrengthAux {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    A.fieldStrength ε = A.fieldStrengthAux ε
```

lemma fieldStrength_basis_repr_eq_single [source]

```
lemma fieldStrength_basis_repr_eq_single {d} {μν : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d)}
    (A : DistElectromagneticPotential d) (ε : 𝓢(SpaceTime d, ℝ)) :
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr (A.fieldStrength ε) μν =
    ((η μν.1 μν.1 * distDeriv μν.1 A ε μν.2) - η μν.2 μν.2 * distDeriv μν.2 A ε μν.1)
```

lemma fieldStrength_diag_zero [source]

```
@[simp]
lemma fieldStrength_diag_zero {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) (μ : Fin 1 ⊕ Fin d) :
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr
    (A.fieldStrength ε) (μ, μ) = 0
```

lemma distDeriv_fieldStrength_diag_zero [source]

```
@[simp]
lemma distDeriv_fieldStrength_diag_zero {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) (μ ν : Fin 1 ⊕ Fin d) :
    (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr
    (distDeriv ν A.fieldStrength ε) (μ, μ) = 0
```

lemma fieldStrength_antisymmetric_basis [source]

```
lemma fieldStrength_antisymmetric_basis {d} (A : DistElectromagneticPotential d)
    (ε : 𝓢(SpaceTime d, ℝ)) (μ ν : Fin 1 ⊕ Fin d) :
    (Vector.basis.tensorProduct Vector.basis).repr
    (A.fieldStrength ε) (μ, ν) = - (Vector.basis.tensorProduct Vector.basis).repr
    (A.fieldStrength ε) (ν, μ)
```

lemma fieldStrength_equivariant [source]

```
lemma fieldStrength_equivariant {d} (A : DistElectromagneticPotential d)
    (Λ : LorentzGroup d) :
    (Λ • A).fieldStrength = Λ • A.fieldStrength
```

2.11 PhysLean.Electromagnetism.Kinematics.MagneticField

PhysLean/Electromagnetism/Kinematics/MagneticField.lean on GitHub.

lemma magneticFieldMatrix_basis_repr_eq_vector_potential [source]

```
lemma magneticFieldMatrix_basis_repr_eq_vector_potential {c : SpeedOfLight}
    (A : DistElectromagneticPotential d)
    (ε : 𝓢(Time × Space d, ℝ)) (i j : Fin d) :
    ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
        (A.magneticFieldMatrix c ε) (i, j) =
      Space.distSpaceDeriv j (A.vectorPotential c) ε i -
      Space.distSpaceDeriv i (A.vectorPotential c) ε j
```

lemma magneticFieldMatrix_distSpaceDeriv_basis_repr_eq_vector_potential [source]

```
lemma magneticFieldMatrix_distSpaceDeriv_basis_repr_eq_vector_potential {c : SpeedOfLight}
    (A : DistElectromagneticPotential d)
    (ε : 𝓢(Time × Space d, ℝ)) (i j k : Fin d) :
    ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
    (Space.distSpaceDeriv k (A.magneticFieldMatrix c) ε) (i, j) =
    Space.distSpaceDeriv k (Space.distSpaceDeriv j (A.vectorPotential c)) ε i -
    Space.distSpaceDeriv k (Space.distSpaceDeriv i (A.vectorPotential c)) ε j
```

lemma magneticFieldMatrix_basis_repr_eq_fieldStrength [source]

```
lemma magneticFieldMatrix_basis_repr_eq_fieldStrength {c : SpeedOfLight}
    (A : DistElectromagneticPotential d)
    (ε : 𝓢(Time × Space d, ℝ)) (i j : Fin d) :
    ((PiLp.basisFun 2 ℝ (Fin d)).tensorProduct (PiLp.basisFun 2 ℝ (Fin d))).repr
        (A.magneticFieldMatrix c ε) (i, j) =
      (Lorentz.Vector.basis.tensorProduct Lorentz.Vector.basis).repr
        (distTimeSlice c A.fieldStrength ε) (Sum.inr i, Sum.inr j)
```

lemma magneticFieldMatrix_one_dim_eq_zero [source]

```
@[simp]
lemma magneticFieldMatrix_one_dim_eq_zero {c : SpeedOfLight}
    (A : DistElectromagneticPotential 1) :
    A.magneticFieldMatrix c = 0
```

2.12 PhysLean.Electromagnetism.PointParticle.OneDimension

PhysLean/Electromagnetism/PointParticle/OneDimension.lean on GitHub.

def oneDimPointParticleCurrentDensity [source]

The current density of a point particle stationary at the origin of 1d space.

```
noncomputable def oneDimPointParticleCurrentDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space 1) :
    DistLorentzCurrentDensity 1
```

lemma oneDimPointParticleCurrentDensity_eq_distTranslate [source]

```
lemma oneDimPointParticleCurrentDensity_eq_distTranslate (c : SpeedOfLight) (q : ℝ) (r₀ : Space
     1) :
    oneDimPointParticleCurrentDensity c q r₀ = ((SpaceTime.distTimeSlice c).symm <|
    constantTime <|
    distTranslate (basis.repr r₀) <|
    ((c * q) • diracDelta' ℝ 0 (Lorentz.Vector.basis (Sum.inl 0))))
```

lemma oneDimPointParticleCurrentDensity_currentDensity [source]

```
@[simp]
lemma oneDimPointParticleCurrentDensity_currentDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space
    1) :
    (oneDimPointParticleCurrentDensity c q r₀).currentDensity c = 0
```

lemma oneDimPointParticleCurrentDensity_chargeDensity [source]

```
@[simp]
lemma oneDimPointParticleCurrentDensity_chargeDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space 1)
     :
    (oneDimPointParticleCurrentDensity c q r₀).chargeDensity c =
    constantTime (q • diracDelta ℝ r₀)
```

lemma oneDimPointParticle_eq_distTranslate [source]

```
lemma oneDimPointParticle_eq_distTranslate (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 1) :
    oneDimPointParticle 𝓕 q r₀ = ((SpaceTime.distTimeSlice 𝓕.c).symm <|
    constantTime <|
    distTranslate (basis.repr r₀) <|
    distOfFunction (fun x => ((- (q * 𝓕.μ₀ * 𝓕.c)/ 2) * ‖x‖) • Lorentz.Vector.basis (Sum.inl
    0))
      (by fun_prop))
```

lemma oneDimPointParticle_electricField_timeDeriv [source]

```
@[simp]
lemma oneDimPointParticle_electricField_timeDeriv (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 1) :
    Space.distTimeDeriv ((oneDimPointParticle 𝓕 q r₀).electricField 𝓕.c) = 0
```

lemma oneDimPointParticle_magneticFieldMatrix [source]

```
lemma oneDimPointParticle_magneticFieldMatrix (q : ℝ) (r₀ : Space 1) :
    (oneDimPointParticle 𝓕 q r₀).magneticFieldMatrix 𝓕.c = 0
```

lemma oneDimPointParticle_div_electricField [source]

```
lemma oneDimPointParticle_div_electricField {𝓕} (q : ℝ) (r₀ : Space 1) :
    distSpaceDiv ((oneDimPointParticle 𝓕 q r₀).electricField 𝓕.c) =
    (𝓕.μ₀ * 𝓕.c ^ 2) • constantTime (q • diracDelta ℝ r₀)
```

lemma oneDimPointParticle_isExterma [source]

```
lemma oneDimPointParticle_isExterma (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 1) :
    (oneDimPointParticle 𝓕 q r₀).IsExtrema 𝓕 (oneDimPointParticleCurrentDensity 𝓕.c q r₀)
```

2.13 PhysLean.Electromagnetism.PointParticle.ThreeDimension

PhysLean/Electromagnetism/PointParticle/ThreeDimension.lean on GitHub.

def threeDimPointParticleCurrentDensity [source]

The current density of a point particle stationary at a point r_0 of 3d space.

```
noncomputable def threeDimPointParticleCurrentDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space 3)
     :
    DistLorentzCurrentDensity 3
```

lemma threeDimPointParticleCurrentDensity_eq_distTranslate [source]

```
lemma threeDimPointParticleCurrentDensity_eq_distTranslate (c : SpeedOfLight) (q : ℝ)
    (r₀ : Space 3) :
    threeDimPointParticleCurrentDensity c q r₀ = ((SpaceTime.distTimeSlice c).symm <|
    constantTime <|
    distTranslate (basis.repr r₀) <|
    ((c * q) • diracDelta' ℝ 0 (Lorentz.Vector.basis (Sum.inl 0))))
```

lemma threeDimPointParticleCurrentDensity_chargeDensity [source]

```
@[simp]
lemma threeDimPointParticleCurrentDensity_chargeDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space
    3) :
    (threeDimPointParticleCurrentDensity c q r₀).chargeDensity c =
    constantTime (q • diracDelta ℝ r₀)
```

lemma threeDimPointParticleCurrentDensity_currentDensity [source]

```
@[simp]
lemma threeDimPointParticleCurrentDensity_currentDensity (c : SpeedOfLight) (q : ℝ) (r₀ : Space
     3) :
    (threeDimPointParticleCurrentDensity c q r₀).currentDensity c = 0
```

def threeDimPointParticle [source]

The electromagnetic potential of a point particle stationary at r_0 of 3d space.

```
noncomputable def threeDimPointParticle (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    DistElectromagneticPotential 3
```

lemma threeDimPointParticle_eq_distTranslate [source]

```
lemma threeDimPointParticle_eq_distTranslate (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    threeDimPointParticle 𝓕 q r₀ = ((SpaceTime.distTimeSlice 𝓕.c).symm <|
    constantTime <|
    distTranslate (basis.repr r₀) <|
    distOfFunction (fun x => (((q * 𝓕.μ₀ * 𝓕.c)/ (4 * π))* ‖x‖⁻¹) •
      Lorentz.Vector.basis (Sum.inl 0))
      ((IsDistBounded.inv.const_mul_fun _).smul_const _))
```

lemma threeDimPointParticle_scalarPotential [source]

```
lemma threeDimPointParticle_scalarPotential (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    (threeDimPointParticle 𝓕 q r₀).scalarPotential 𝓕.c =
    Space.constantTime (distOfFunction (fun x => (q/ (4 * π * 𝓕.ε₀))• ‖x - r₀‖⁻¹)
      (((IsDistBounded.inv_shift _).const_mul_fun _)))
```

lemma threeDimPointParticle_vectorPotential [source]

```
@[simp]
lemma threeDimPointParticle_vectorPotential (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    (threeDimPointParticle 𝓕 q r₀).vectorPotential 𝓕.c = 0
```

lemma threeDimPointParticle_electricField [source]

```
lemma threeDimPointParticle_electricField (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    (threeDimPointParticle 𝓕 q r₀).electricField 𝓕.c =
    (q/ (4 * π * 𝓕.ε₀)) • constantTime (distOfFunction (fun x : Space 3 =>
      ‖x - r₀‖ ^ (- 3 : ℤ) • basis.repr (x - r₀))
      ((IsDistBounded.zpow_smul_repr_self (- 3 : ℤ) (by omega)).comp_sub_right r₀))
```

lemma threeDimPointParticle_electricField_timeDeriv [source]

```
@[simp]
lemma threeDimPointParticle_electricField_timeDeriv (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    Space.distTimeDeriv ((threeDimPointParticle 𝓕 q r₀).electricField 𝓕.c) = 0
```

lemma threeDimPointParticle_magneticFieldMatrix [source]

```
@[simp]
lemma threeDimPointParticle_magneticFieldMatrix (q : ℝ) (r₀ : Space 3) :
    (threeDimPointParticle 𝓕 q r₀).magneticFieldMatrix 𝓕.c = 0
```

lemma threeDimPointParticle_div_electricField [source]

```
lemma threeDimPointParticle_div_electricField {𝓕} (q : ℝ) (r₀ : Space 3) :
    distSpaceDiv ((threeDimPointParticle 𝓕 q r₀).electricField 𝓕.c) =
    (1/𝓕.ε₀) • constantTime (q • diracDelta ℝ r₀)
```

lemma threeDimPointParticle_isExterma [source]

```
lemma threeDimPointParticle_isExterma (𝓕 : FreeSpace) (q : ℝ) (r₀ : Space 3) :
    (threeDimPointParticle 𝓕 q r₀).IsExtrema 𝓕 (threeDimPointParticleCurrentDensity 𝓕.c q r₀)
```

2.14 PhysLean.Mathematics.List

PhysLean/Mathematics/List.lean on GitHub.

lemma insertionSortEquiv_congr_apply [source]

```
lemma insertionSortEquiv_congr_apply {α : Type} {r : α → α → Prop} [DecidableRel r] (l l' :
    List α)
    (h : l = l') (i : Fin l.length) :
    insertionSortEquiv r l i =
    Fin.cast (by simp [h])
    ((insertionSortEquiv r l') (Fin.cast (by simp [h]) i))
```

2.15 PhysLean.Particles.BeyondTheStandardModel.TwoHDM.Basic

PhysLean/Particles/BeyondTheStandardModel/TwoHDM/Basic.lean on GitHub.

structure TwoHiggsDoublet [source]

The configuration space of the two Higgs doublet model.

```
structure TwoHiggsDoublet where
  Φ1 : HiggsVec
  Φ2 : HiggsVec
```

2.16 PhysLean.QuantumMechanics.OneDimension.HarmonicOscillator.Basic

PhysLean/QuantumMechanics/OneDimension/HarmonicOscillator/Basic.lean on GitHub.

lemma schrodingerOperator_add [source]

The Schrodinger operator is additive.

```
lemma schrodingerOperator_add (ψ φ : ℝ → ℂ)
    (hψ : Differentiable ℝ ψ) (hψ' : Differentiable ℝ (deriv ψ))
    (hφ : Differentiable ℝ φ) (hφ' : Differentiable ℝ (deriv φ)) :
    Q.schrodingerOperator (ψ + φ) = Q.schrodingerOperator ψ + Q.schrodingerOperator φ
```

lemma schrodingerOperator_smul [source]

The Schrodinger operator is homogeneous.

```
lemma schrodingerOperator_smul (c : ℂ) (ψ : ℝ → ℂ)
    (hψ : Differentiable ℝ ψ) (hψ' : Differentiable ℝ (deriv ψ)) :
    Q.schrodingerOperator (c • ψ) = c • Q.schrodingerOperator ψ
```

2.17 PhysLean.Relativity.CliffordAlgebra

PhysLean/Relativity/CliffordAlgebra.lean on GitHub.

lemma γ_subtype_in_range [source]

Each gamma matrix (as an element of diracAlgebra) is in the range of ofCliffordAlgebra.

```
lemma γ_subtype_in_range (i : Fin 4) :
    ⟨γ i, γ_in_diracAlgebra i⟩ ∈ ofCliffordAlgebra.range
```

lemma mem_adjoin_imp_subtype_in_range [source]

Helper lemma: If a matrix is in Algebra.adjoin ℝ γSet, then its subtype is in the range.

```
private lemma mem_adjoin_imp_subtype_in_range (m : Matrix (Fin 4) (Fin 4) ℂ)
    (hm : m ∈ Algebra.adjoin ℝ γSet) : ⟨m, hm⟩ ∈ ofCliffordAlgebra.range
```

lemma ofCliffordAlgebra_range_eq_top [source]

The range of ofCliffordAlgebra equals the top subalgebra (i.e., all of diracAlgebra).

```
lemma ofCliffordAlgebra_range_eq_top : ofCliffordAlgebra.range = ⊤
```

theorem ofCliffordAlgebra_surjective [source]

The homomorphism `ofCliffordAlgebra` is surjective.

```
theorem ofCliffordAlgebra_surjective : Function.Surjective ofCliffordAlgebra
```

2.18 PhysLean.Relativity.LorentzAlgebra.Basis

PhysLean/Relativity/LorentzAlgebra/Basis.lean on GitHub.

def boostGenerator [source]

The boost generator K_i in the Lorentz algebra so(1,3).

This matrix generates infinitesimal Lorentz boosts in the i-th spatial direction. The matrix has non-zero entries only at positions (0, i+1) and (i+1, 0) with value 1, where we use the index convention 0 = time, 1,2,3 = space.

Properties - Symmetric: $K\_i^T = K\_i$ - Traceless: tr(K_i) = 0 - Satisfies Lorentz algebra condition: $K\_i^T \eta = -\eta K\_i$

Physical Meaning Exponentiating $\beta \cdot K\_i$ produces a finite Lorentz boost with rapidity β in direction i.

```
def boostGenerator (i : Fin 3) : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ
```

def rotationGenerator [source]

The rotation generator J_i in the Lorentz algebra so(1,3).

This matrix generates infinitesimal rotations about the i-th axis following the right-hand rule. The matrix acts only on spatial indices in the antisymmetric pattern characteristic of angular momentum generators.

Properties - Antisymmetric: $J\_i^T = -J\_i$ - Traceless: tr(J_i) = 0 - Satisfies Lorentz algebra condition: $J\_i^T \eta = -\eta J\_i$

Structure - J_0 (rotation about x-axis): Acts on (y,z) components - J_1 (rotation about y-axis): Acts on (z,x) components - J_2 (rotation about z-axis): Acts on (x,y) components

Physical Meaning Exponentiating $\theta \cdot J\_i$ produces a finite rotation by angle θ about axis i.

```
def rotationGenerator (i : Fin 3) : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ
```

lemma boostGenerator_mem [source]

The boost generator K_i is in the Lorentz algebra.

```
lemma boostGenerator_mem (i : Fin 3) : boostGenerator i ∈ lorentzAlgebra
```

lemma rotationGenerator_mem [source]

The rotation generator J_i is in the Lorentz algebra.

```
lemma rotationGenerator_mem (i : Fin 3) : rotationGenerator i ∈ lorentzAlgebra
```

2.19 PhysLean.Relativity.MinkowskiMatrix

PhysLean/Relativity/MinkowskiMatrix.lean on GitHub.

lemma as_diagonal [source]

The Minkowski matrix as a diagonal matrix.

```
lemma as_diagonal : @minkowskiMatrix d = diagonal (Sum.elim 1 (-1))
```

2.20 PhysLean.Relativity.Tensors.Basic

PhysLean/Relativity/Tensors/Basic.lean on GitHub.

lemma symm [source]

```
lemma PermCond.symm {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    {σ : Fin m → Fin n} (h : PermCond c c1 σ) :
    PermCond c1 c (h.inv σ)
```

lemma permT_eq_zero_iff [source]

```
lemma permT_eq_zero_iff {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    {σ : Fin m → Fin n} (h : PermCond c c1 σ) (t : S.Tensor c) :
    permT σ h t = 0 ↔ t = 0
```

2.21 PhysLean.SpaceAndTime.Space.Basic

PhysLean/SpaceAndTime/Space/Basic.lean on GitHub.

lemma point_dim_zero_eq [source]

```
lemma point_dim_zero_eq (p : Space 0) : p = 0
```

lemma volume_metricBall_two [source]

```
@[simp]
lemma volume_metricBall_two :
    volume (Metric.ball (0 : Space 2) 1) = ENNReal.ofReal Real.pi
```

lemma volume_metricBall_two_real [source]

```
@[simp]
lemma volume_metricBall_two_real :
    (volume.real (Metric.ball (0 : Space 2) 1)) = Real.pi
```

2.22 PhysLean.SpaceAndTime.Space.IsDistBounded

PhysLean/SpaceAndTime/Space/IsDistBounded.lean on GitHub.

lemma inv_shift [source]

```
@[fun_prop]
lemma inv_shift {d : ℕ}
    (g : Space d.succ.succ) :
    IsDistBounded (d := d.succ.succ) (fun x => ‖x - g‖⁻¹)
```

lemma norm_sub [source]

```
@[fun_prop]
lemma norm_sub {d : ℕ} (g : Space d) :
    IsDistBounded (d := d) (fun x => ‖x - g‖)
```

lemma norm_add [source]

```
@[fun_prop]
lemma norm_add {d : ℕ} (g : Space d) :
    IsDistBounded (d := d) (fun x => ‖x + g‖)
```

lemma zpow_smul_repr_self_sub [source]

```
lemma zpow_smul_repr_self_sub {d : ℕ} (n : ℤ) (hn : - (d - 1 : ℕ) - 1 ≤ n)
    (y : Space d) :
    IsDistBounded (d := d) (fun x => ‖x - y‖ ^ n • basis.repr (x - y))
```

2.23 PhysLean.SpaceAndTime.Space.Translations

PhysLean/SpaceAndTime/Space/Translations.lean on GitHub.

def distTranslate [source]

The continuous linear map translating distributions.

```
noncomputable def distTranslate {d : ℕ} (a : EuclideanSpace ℝ (Fin d)) :
    ((Space d) →d[ℝ] X) →ₗ[ℝ] ((Space d) →d[ℝ] X) where
```

lemma distTranslate_apply [source]

```
lemma distTranslate_apply {d : ℕ} (a : EuclideanSpace ℝ (Fin d))
    (T : (Space d) →d[ℝ] X) (η : 𝓢(Space d, ℝ)) :
    distTranslate a T η = T (translateSchwartz (-a) η)
```

lemma distTranslate_distGrad [source]

```
@[simp]
lemma distTranslate_distGrad {d : ℕ} (a : EuclideanSpace ℝ (Fin d))
    (T : (Space d) →d[ℝ] ℝ) :
    distGrad (distTranslate a T) = distTranslate a (distGrad T)
```

lemma distTranslate_ofFunction [source]

```
lemma distTranslate_ofFunction {d : ℕ} (a : EuclideanSpace ℝ (Fin d.succ))
    (f : Space d.succ → X) (hf : IsDistBounded f) :
```

```
    distTranslate a (distOfFunction f hf) =
    distOfFunction (fun x => f (x - basis.repr.symm a))
    (IsDistBounded.comp_add_right hf (- basis.repr.symm a))
```

lemma distDiv_distTranslate [source]

```
@[simp]
lemma distDiv_distTranslate {d : ℕ} (a : EuclideanSpace ℝ (Fin d))
    (T : (Space d) →d[ℝ] EuclideanSpace ℝ (Fin d)) :
    distDiv (distTranslate a T) = distTranslate a (distDiv T)
```

2.24 PhysLean.SpaceAndTime.SpaceTime.Derivatives

PhysLean/SpaceAndTime/SpaceTime/Derivatives.lean on GitHub.

lemma deriv_equivariant [source]

```
lemma deriv_equivariant (f : SpaceTime d → M) (Λ : LorentzGroup d) (x : SpaceTime d)
    (hf : Differentiable ℝ f) (μ : Fin 1 ⊕ Fin d) :
    ∂_ μ (fun x => Λ • f (Λ⁻¹ • x)) x =
    ∑ ν, Λ⁻¹.1 ν μ • Λ • ∂_ ν f (Λ⁻¹ • x)
```

lemma distDeriv_apply' [source]

```
lemma distDeriv_apply' {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (μ : Fin 1 ⊕ Fin d) (f : (SpaceTime d) →d[ℝ] M) (ε : 𝓢(SpaceTime d, ℝ)) :
    distDeriv μ f ε =
    - f ((SchwartzMap.evalCLM (𝕜 := ℝ) (Lorentz.Vector.basis μ)) ((fderivCLM ℝ) ε))
```

lemma apply_fderiv_eq_distDeriv [source]

```
lemma apply_fderiv_eq_distDeriv {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (μ : Fin 1 ⊕ Fin d) (f : (SpaceTime d) →d[ℝ] M) (ε : 𝓢(SpaceTime d, ℝ)) :
    f ((SchwartzMap.evalCLM (𝕜 := ℝ) (Lorentz.Vector.basis μ)) ((fderivCLM ℝ) ε)) =
    - distDeriv μ f ε
```

def tensorDeriv [source]

The derivative of a tensor, as a tensor.

```
def tensorDeriv (f : SpaceTime d → M) :
    SpaceTime d → Lorentz.CoVector d ⊗[ℝ] M
```

lemma tensorDeriv_equivariant [source]

```
lemma tensorDeriv_equivariant (f : SpaceTime d → M) (Λ : LorentzGroup d) (x : SpaceTime d)
    (hf : Differentiable ℝ f) :
    tensorDeriv (fun x => Λ • f (Λ⁻¹ • x)) x =
    Λ • tensorDeriv f (Λ⁻¹ • x)
```

lemma tensorDeriv_toTensor_basis_repr [source]

```
lemma tensorDeriv_toTensor_basis_repr
    {f : SpaceTime d → M}
    (hf : Differentiable ℝ f) (x : SpaceTime d)
    (b : Tensor.ComponentIdx (Fin.append ![realLorentzTensor.Color.down] c)) :
    (Tensor.basis _).repr (Tensorial.toTensor (tensorDeriv f x)) b =
    ∂_ (Lorentz.CoVector.indexEquiv (Tensor.ComponentIdx.prodEquiv b).1)
      (fun x => (Tensor.basis _).repr (Tensorial.toTensor (f x))
        (Tensor.ComponentIdx.prodEquiv b).2) x
```

def distTensorDeriv [source]

The derivative of a tensor, as a tensor for distributions.

```
def distTensorDeriv {M d} [NormedAddCommGroup M]
    [InnerProductSpace ℝ M] [FiniteDimensional ℝ M] :
    ((SpaceTime d) →d[ℝ] M) →ₗ[ℝ] ((SpaceTime d) →d[ℝ] Lorentz.CoVector d ⊗[ℝ] M) where
```

lemma distTensorDeriv_apply [source]

```
lemma distTensorDeriv_apply {M d} [NormedAddCommGroup M]
    [InnerProductSpace ℝ M] [FiniteDimensional ℝ M] (f : (SpaceTime d) →d[ℝ] M)
    (ε : 𝓢(SpaceTime d, ℝ)) :
    distTensorDeriv f ε = ∑ μ, (Lorentz.CoVector.basis μ) ⊗ₜ distDeriv μ f ε
```

lemma distTensorDeriv_equivariant [source]

```
lemma distTensorDeriv_equivariant {M : Type} [NormedAddCommGroup M]
    [InnerProductSpace ℝ M] [FiniteDimensional ℝ M] [(realLorentzTensor d).Tensorial c M]
    (f : (SpaceTime d) →d[ℝ] M) (Λ : LorentzGroup d) :
    distTensorDeriv (Λ • f) = Λ • distTensorDeriv f
```

lemma distTensorDeriv_toTensor_basis_repr [source]

```
lemma distTensorDeriv_toTensor_basis_repr {M : Type} [NormedAddCommGroup M]
    [InnerProductSpace ℝ M] [FiniteDimensional ℝ M] [(realLorentzTensor d).Tensorial c M]
    {f : (SpaceTime d) →d[ℝ] M}
    (ε : 𝓢(SpaceTime d, ℝ))
    (b : Tensor.ComponentIdx (Fin.append ![realLorentzTensor.Color.down] c)) :
    (Tensor.basis _).repr (Tensorial.toTensor (distTensorDeriv f ε)) b =
    (Tensor.basis _).repr (Tensorial.toTensor
    (distDeriv (Lorentz.CoVector.indexEquiv (Tensor.ComponentIdx.prodEquiv b).1) f ε))
    (Tensor.ComponentIdx.prodEquiv b).2
```

2.25 PhysLean.SpaceAndTime.SpaceTime.LorentzAction

PhysLean/SpaceAndTime/SpaceTime/LorentzAction.lean on GitHub.

def distActionLinearMap [source]

The Lorentz action on distributions as a linear map.

```
def distActionLinearMap {d} {M : Type} [NormedAddCommGroup M]
    [NormedSpace ℝ M] [Tensorial (realLorentzTensor d) c M] [T2Space M](Λ : LorentzGroup d) :
```

```
    ((SpaceTime d) →d[ℝ] M) →ₗ[ℝ] ((SpaceTime d) →d[ℝ] M) where
```

2.26 PhysLean.SpaceAndTime.SpaceTime.TimeSlice

PhysLean/SpaceAndTime/SpaceTime/TimeSlice.lean on GitHub.

lemma distTimeSlice_symm_distTimeDeriv_eq [source]

```
lemma distTimeSlice_symm_distTimeDeriv_eq {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {c : SpeedOfLight}
    (f : (Time × Space d) →d[ℝ] M) :
    (distTimeSlice c).symm (Space.distTimeDeriv f) =
    c.val • distDeriv (Sum.inl 0) ((distTimeSlice c).symm f)
```

2.27 PhysLean.SpaceAndTime.TimeAndSpace.Basic

PhysLean/SpaceAndTime/TimeAndSpace/Basic.lean on GitHub.

lemma distTimeDeriv_apply' [source]

```
lemma distTimeDeriv_apply' {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (f : (Time × Space d) →d[ℝ] M) (ε : 𝓢(Time × Space d, ℝ)) :
    (distTimeDeriv f) ε = -f (SchwartzMap.evalCLM (𝕜 := ℝ) (1, 0) ((fderivCLM ℝ) ε))
```

lemma apply_fderiv_eq_distTimeDeriv [source]

```
lemma apply_fderiv_eq_distTimeDeriv {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (f : (Time × Space d) →d[ℝ] M) (ε : 𝓢(Time × Space d, ℝ)) :
    f (SchwartzMap.evalCLM (𝕜 := ℝ) (1, 0) ((fderivCLM ℝ) ε)) = - (distTimeDeriv f) ε
```

lemma distSpaceDeriv_apply' [source]

```
lemma distSpaceDeriv_apply' {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (i : Fin d) (f : (Time × Space d) →d[ℝ] M) (ε : 𝓢(Time × Space d, ℝ)) :
    (distSpaceDeriv i f) ε =
    - f ((SchwartzMap.evalCLM (𝕜 := ℝ) (0, basis i)) ((fderivCLM ℝ) ε))
```

lemma apply_fderiv_eq_distSpaceDeriv [source]

```
lemma apply_fderiv_eq_distSpaceDeriv {M d} [NormedAddCommGroup M] [NormedSpace ℝ M]
    (i : Fin d) (f : (Time × Space d) →d[ℝ] M) (ε : 𝓢(Time × Space d, ℝ)) :
    f ((SchwartzMap.evalCLM (𝕜 := ℝ) (0, basis i)) ((fderivCLM ℝ) ε)) =
    - (distSpaceDeriv i f) ε
```

2.28 PhysLean.StatisticalMechanics.CanonicalEnsemble.TwoState

PhysLean/StatisticalMechanics/CanonicalEnsemble/TwoState.lean on GitHub.

instance IsFinite (twoState E₀ E₁) [source]

```
instance {E₀ E₁} : IsFinite (twoState E₀ E₁) where
```

lemma twoState_partitionFunction_apply [source]

```
lemma twoState_partitionFunction_apply (E₀ E₁ : ℝ) (T : Temperature) :
    (twoState E₀ E₁).partitionFunction T = exp (- β T * E₀) + exp (- β T * E₁)
```

lemma twoState_probability_fst [source]

Probability of the first state (energy `E₀`) in closed form.

```
lemma twoState_probability_fst (E₀ E₁ : ℝ) (T : Temperature) :
    (twoState E₀ E₁).probability T 0 = 1 / 2 * (1 + Real.tanh (β T * (E₁ - E₀) / 2))
```

2.29 PhysLean.Thermodynamics.IdealGas.Basic

PhysLean/Thermodynamics/IdealGas/Basic.lean on GitHub.

def entropy [source]

Entropy of a monophase ideal gas: S(U,V,N) = N s0 + N R (c log(U/U0) + log(V/V0) - (c+1) log(N/N0)).

```
def entropy
    (c R s0 U0 V0 N0 : ℝ) (U V N : ℝ) : ℝ
```

theorem adiabatic_relation_log [source]

*Adiabatic relation in logarithmic form: If S(Ua,Va,N) = S(Ub,Vb,N) with N fixed, then c * log (Ua/Ub) + log (Va/Vb) = 0.*

```
theorem adiabatic_relation_log
    {s0 U0 V0 N0 c R : ℝ}
    {Ua Ub Va Vb N : ℝ}
    (hUa : 0 < Ua) (hUb : 0 < Ub)
    (hVa : 0 < Va) (hVb : 0 < Vb)
    (hN : 0 < N)
    (hU0 : 0 < U0) (hV0 : 0 < V0)
    (hR : 0 < R)
    (hS :
      entropy c R s0 U0 V0 N0 Ua Va N =
      entropy c R s0 U0 V0 N0 Ub Vb N) :
    c * log (Ua / Ub) + log (Va / Vb) = 0
```

theorem adiabatic_relation_UaUbVaVb [source]

*Adiabatic relation in product form: If S(Ua,Va,N) = S(Ub,Vb,N) with N fixed, then (Ua/Ub)^c * (Va/Vb) = 1.*

```
theorem adiabatic_relation_UaUbVaVb
    {s0 U0 V0 N0 c R : ℝ}
    {Ua Ub Va Vb N : ℝ}
    (hUa : 0 < Ua) (hUb : 0 < Ub)
    (hVa : 0 < Va) (hVb : 0 < Vb)
    (hN : 0 < N)
    (hU0 : 0 < U0) (hV0 : 0 < V0)
    (hR : 0 < R)
    (hS :
```

```
    entropy c R s0 U0 V0 N0 Ua Va N =
    entropy c R s0 U0 V0 N0 Ub Vb N) :
  (Real.rpow (Ua / Ub) c) * (Va / Vb) = 1
```